



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE ESTUDIOS
PROFESIONALES
ZONA MEDIA

INGENIERÍA MECATRÓNICA



PRÁCTICA 12

Control de LED RGB Mediante Señales PWM

MATERIA: MICROCONTROLADORES

DOCENTE: Ing. JESÚS PADRÓN

ALUMNO: CARLOS ANGEL GARCÍA SIFUENTES

CLAVE: A383917

CARRERA: INGENIERÍA MECATRÓNICA

SEMESTRE: 4° SEMESTRE

FECHA: 24 DE MAYO DE 2026

Control de Color en LED RGB mediante Modulación por Ancho de Pulso (PWM) y Lectura Analógica de Potenciómetros en RPi Pico W

CARLOS ÁNGEL GARCÍA SIFUENTES

a383917@alumnos.uaslp.mx

Fecha: 20 de mayo de 2026

Resumen—Se presenta el desarrollo de la Práctica 12, que introduce la Modulación por Ancho de Pulso (PWM) como técnica de control analógico sobre un LED RGB. Tres potenciómetros conectados a los canales ADC0, ADC1 y ADC2 de la Raspberry Pi Pico W (pines GP26, GP27, GP28) proporcionan valores analógicos que se escalan a ciclos de trabajo de 0 a 255 y se aplican a los canales PWM de los pines GP6, GP10 y GP14, controlando la intensidad de los componentes rojo, verde y azul del LED RGB respectivamente. El código en C utiliza la librería `hardware_pwm` del SDK, configura una resolución de 8 bits (256 niveles) por canal y actualiza el monitor serial con los porcentajes de cada canal cada 500 ms.

I. INTRODUCCIÓN

I-A Objetivo e Importancia del PWM

En electrónica digital, los microcontroladores solo pueden generar señales de salida en dos estados: alto (3.3 V) o bajo (0 V). Sin embargo, muchas aplicaciones requieren controlar magnitudes analógicas como el brillo de un LED, la velocidad de un motor o la posición de un servo. La **Modulación por Ancho de Pulso** (PWM, *Pulse Width Modulation*) es la técnica estándar para simular una salida analógica desde una salida digital: la señal alterna entre alto y bajo a una frecuencia fija, variando el tiempo que permanece en alto (ciclo de trabajo) para controlar la potencia promedio entregada al dispositivo.

Su importancia radica en que es una solución eficiente en energía, no disipa potencia como lo haría un divisor resistivo, y es nativamente soportada por el hardware del microcontrolador sin carga adicional para el procesador.

I-B Señal PWM y Ciclo de Trabajo

Una señal PWM se caracteriza por dos parámetros:

- **Período** (T): el tiempo de un ciclo completo, determinado por la frecuencia de la señal.
- **Ciclo de trabajo** (*duty cycle*, D): la fracción del

período en que la señal permanece en estado alto, expresada como porcentaje:

$$D = \frac{t_{ON}}{T} \times 100 \% \quad (1)$$

Un ciclo de trabajo del 0 % mantiene la salida siempre en bajo (LED apagado), el 50 % produce un brillo medio, y el 100 % mantiene la salida siempre en alto (LED a máxima intensidad).

I-C LED RGB y Mezcla Aditiva de Color

Un LED RGB contiene tres diodos independientes dentro de un mismo encapsulado: rojo (R), verde (G) y azul (B). Controlando la intensidad de cada canal mediante PWM se logra cualquier color del espacio RGB por mezcla aditiva de luz. La Tabla I muestra algunos ejemplos.

Tabla I: Ejemplos de mezcla aditiva de color RGB con PWM

Color resultante	R (%)	G (%)	B (%)
Rojo puro	100	0	0
Verde puro	0	100	0
Azul puro	0	0	100
Amarillo	100	100	0
Cian	0	100	100
Magenta	100	0	100
Blanco	100	100	100
Apagado	0	0	0

I-D LED RGB de Ánodo Común vs. Cátodo Común

Esta distinción es crítica para la lógica del código. En un LED RGB de **cátodo común**, el terminal compartido va a GND y cada canal se enciende con un nivel alto. En un LED RGB de **ánodo común**, el terminal compartido va a 3.3 V y cada canal se enciende con un nivel *bajo*: el ciclo de trabajo se invierte. El código de esta práctica contempla ánodo común: aplica `255 - duty` antes de escribir el PWM, de modo que un potenciómetro al 100 %

(duty = 255) produce $255 - 255 = 0$ (canal encendido al máximo).

II. DESARROLLO

II-A Material y Recursos Utilizados

- **Hardware:** 1 Placa Raspberry Pi Pico W, 1 cable micro USB tipo B, 1 LED RGB (ánodo común), 3 potenciómetros de 10 kΩ, 3 resistencias de 100 Ω (protección de canales RGB), 1 protoboard y alambre de conexión.
- **Software y librerías:** SDK de Raspberry Pi Pico, Pico – Visual Studio Code, librerías `pico_stdlib`, `hardware_adc` y `hardware_pwm`.

II-B Asignación de Pines

Tabla II: Asignación de pines GPIO para la Práctica 12

Canal	Potenciómetro	Pin ADC	Pin PWM (L
Rojo (R)	POT R	GP26 (ADC0)	GP6
Verde (G)	POT G	GP27 (ADC1)	GP10
Azul (B)	POT B	GP28 (ADC2)	GP14

II-C Estructura del Proyecto y Configuración de CMake

Una novedad en esta práctica es la inclusión de `hardware_pwm` como dependencia adicional en el `CMakeLists.txt`, que proporciona las funciones de control PWM del SDK.

ARCHIVO CMakeLists.txt

```
cmake_minimum_required(VERSION 3.13)
set(PICO_BOARD pico_w)
include(pico_sdk_import.cmake)

project(PRACTICA_12)

pico_sdk_init()

add_executable(RGB
    RGB.c
)

target_link_libraries(
    RGB
    pico_stdlib
    hardware_adc
    hardware_pwm
)

pico_enable_stdio_usb(1)
```

```
pico_enable_stdio_uart(0)
```

```
pico_add_extra_outputs(1)
```

II-D Implementación del Código en C

El archivo `RGB.c` organiza el programa en dos funciones auxiliares (`setup_pwm` y `set_pwm`) y un lazo principal que lee los tres ADC, escala los valores y los aplica a los tres canales PWM.

CÓDIGO FUENTE – RGB.c

```
#include "pico/stdlib.h"
#include "hardware/adc.h"
#include "hardware/pwm.h"
#include <stdio.h>

#define PIN_R 6 // PWM rojo - GP6
#define PIN_G 10 // PWM verde - GP10
#define PIN_B 14 // PWM azul - GP14

#define ADC_R 0 // GP26
#define ADC_G 1 // GP27
#define ADC_B 2 // GP28

// Configura un pin como salida PWM con
// wrap = 255 (8 bits)
static void setup_pwm(uint pin) {
    gpio_set_function(pin, GPIO_FUNC_PWM);
    uint slice = pwm_gpio_to_slice_num(pin);
    ;
    pwm_set_wrap(slice, 255);
    pwm_set_enabled(slice, true);
}

// Escribe duty cycle (0-255) en un pin PWM
static void set_pwm(uint pin, uint8_t duty)
{
    uint slice = pwm_gpio_to_slice_num(
        pin);
    uint channel = pwm_gpio_to_channel(pin);
    ;
    pwm_set_chan_level(slice, channel, duty);
    ;
}

int main() {
    stdio_init_all();

    // Inicializar ADC en los tres canales
    adc_init();
    adc_gpio_init(26);
    adc_gpio_init(27);
    adc_gpio_init(28);

    // Inicializar PWM en los tres pines
    // del LED RGB
```

```

setup_pwm(PIN_R);
setup_pwm(PIN_G);
setup_pwm(PIN_B);

while (true) {
    // 1. Leer canales ADC (0 a 4095)
    adc_select_input(ADC_R);
    uint16_t raw_r = adc_read();
    adc_select_input(ADC_G);
    uint16_t raw_g = adc_read();
    adc_select_input(ADC_B);
    uint16_t raw_b = adc_read();

    // 2. Convertir a porcentaje para
    // monitor serial (0-100)
    uint8_t pct_r = (raw_r * 100) /
        4095;
    uint8_t pct_g = (raw_g * 100) /
        4095;
    uint8_t pct_b = (raw_b * 100) /
        4095;

    // 3. Convertir a duty cycle PWM
    // (0-255)
    uint8_t duty_r = (raw_r * 255) /
        4095;
    uint8_t duty_g = (raw_g * 255) /
        4095;
    uint8_t duty_b = (raw_b * 255) /
        4095;

    // 4. Aplicar al LED RGB (invertido
    // para anodo comun)
    set_pwm(PIN_R, 255 - duty_r);
    set_pwm(PIN_G, 255 - duty_g);
    set_pwm(PIN_B, 255 - duty_b);

    // 5. Imprimir en monitor serial
    // cada 500 ms
    static uint32_t last = 0;
    uint32_t now = to_ms_since_boot(
        get_absolute_time());
    if (now - last >= 500) {
        last = now;
        printf("R: %d%% | G: %d%% | B:
            %d%%\n",
            pct_r, pct_g, pct_b);
    }

    sleep_ms(10); // Estabilidad del
    ADC
}

return 0;
}

```

II-D1. Análisis de `setup_pwm()`

Esta función configura un pin GPIO como salida PWM en tres pasos. Primero, `gpio_set_function(pin, GPIO_FUNC_PWM)` asigna la función PWM al pin,

desconectando su buffer GPIO estándar. Luego, `pwm_gpio_to_slice_num(pin)` obtiene el número de *slice* (bloque PWM del RP2040) asociado al pin, y `pwm_set_wrap(slice, 255)` fija el valor máximo del contador en 255, lo que da una resolución de 8 bits (256 niveles de duty cycle). Finalmente, `pwm_set_enabled(slice, true)` arranca el contador PWM.

II-D2. Cadena de conversión ADC → PWM

Por cada canal la cadena de transformación es:

$$\text{duty} = \left\lfloor \frac{N_{ADC} \times 255}{4095} \right\rfloor \quad (2)$$

donde $N_{ADC} \in [0, 4095]$ es el valor leído del potenciómetro. Para el LED de ánodo común se aplica la inversión:

$$\text{duty}_{\text{aplicado}} = 255 - \text{duty} \quad (3)$$

Esto garantiza que cuando el potenciómetro está al máximo ($\text{duty} = 255$), el canal recibe 0 (encendido al 100 % en ánodo común), y cuando está al mínimo ($\text{duty} = 0$), el canal recibe 255 (apagado).

II-D3. Impresión no bloqueante en monitor serial

El código usa una variable estática `last` y `to_ms_since_boot()` para imprimir los porcentajes solo cuando hayan transcurrido 500 ms desde la última impresión, sin detener el lazo con `sleep_ms(500)`. Esto permite que el PWM se actualice con cada iteración del lazo (10 ms) mientras el monitor serial recibe datos a 2 Hz, logrando una respuesta visual inmediata al girar los potenciómetros.

II-E Diagrama de Conexiones

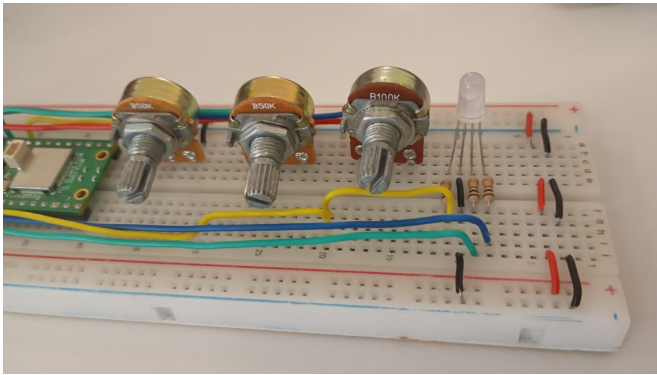


Figura 1: Esquemático del circuito: tres potenciómetros en GP26–28 (ADC), LED RGB de ánodo común en GP6, GP10, GP14 (PWM) con resistencias de protección de 100 Ω .

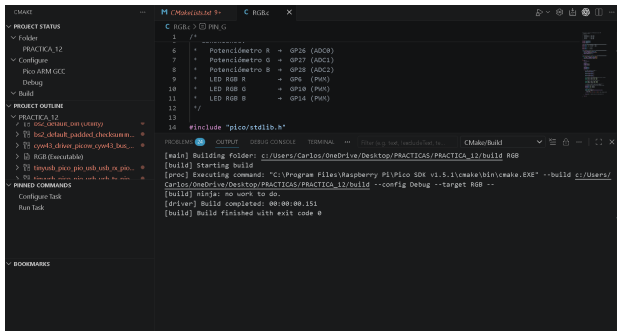


Figura 2: Compilación exitosa del proyecto en Pico Visual Studio Code (Build finished with exit code 0).

III. PREGUNTAS DE REPASO

III-A ¿Qué es el ciclo de trabajo en un PWM?

El ciclo de trabajo (*duty cycle*) es la fracción del período de la señal PWM durante la cual la salida permanece en estado alto, expresada como porcentaje según la ecuación (1). Representa directamente la potencia promedio entregada a la carga: un ciclo de trabajo del 0 % no entrega energía, mientras que uno del 100 % entrega la potencia máxima. En el contexto de un LED, el ciclo de trabajo determina el brillo percibido por el ojo humano gracias a la integración temporal de los destellos de alta frecuencia.

III-B ¿Qué significa que el ciclo de trabajo del PWM sea de 40 %?

Un ciclo de trabajo del 40 % significa que en cada período de la señal PWM, la salida permanece en estado alto durante el 40 % del tiempo y en estado bajo durante

el 60 % restante. Por ejemplo, si la frecuencia es 1 kHz (período de 1 ms), la señal estaría en alto durante 0.4 ms y en bajo durante 0.6 ms. En términos de potencia promedio entregada a un LED de 3.3 V, equivaldría aproximadamente a 1.32 V efectivos, produciendo un brillo del 40 % de la intensidad máxima.

III-C ¿Cómo se genera una señal PWM en un microcontrolador?

En el RP2040 de la Raspberry Pi Pico W, el hardware PWM funciona mediante un contador ascendente que se incrementa con cada ciclo del reloj del sistema. Cuando el contador alcanza el valor de comparación (*level*), la salida cambia de alto a bajo; cuando alcanza el valor de envolvente (*wrap*), el contador regresa a cero y la salida vuelve a alto, reiniciando el ciclo. En el código se configura con `pwm_set_wrap(slice, 255)` (resolución de 8 bits) y `pwm_set_chan_level(slice, channel, duty)` para fijar el punto de comparación. El resultado es una señal PWM de frecuencia constante con ciclo de trabajo variable.

III-D ¿Qué efecto tiene variar la frecuencia de la señal PWM en la iluminación del LED RGB?

A frecuencias muy bajas (por debajo de 50 Hz), el ojo humano puede percibir el parpadeo individual del LED, lo que resulta molesto e inapropiado para iluminación. A frecuencias medias (100 Hz–1 kHz), el parpadeo ya no es visible y el brillo percibido depende únicamente del ciclo de trabajo. A frecuencias muy altas (>100 kHz), la inductancia y capacitancia parásita del circuito pueden distorsionar la forma de la señal, reduciendo la eficiencia del control. La frecuencia óptima para control de LEDs está típicamente entre 1 kHz y 20 kHz.

III-E ¿Cuál es la relación entre la resolución del ADC y la precisión del control PWM?

En este sistema, el ADC de 12 bits (4096 niveles) se escala a un PWM de 8 bits (256 niveles). El eslabón más débil de la cadena es el PWM: aunque el ADC puede distinguir 4096 posiciones del potenciómetro, el LED solo puede reproducir 256 intensidades distintas. Cada nivel de PWM representa aproximadamente $4095/255 \approx 16$ cuentas ADC, lo que significa que pequeñas variaciones del potenciómetro (menores a $1/256$ del recorrido total) no producen cambio visible en el LED. Para aprovechar completamente la resolución del ADC de 12 bits se requeriría un PWM de igual resolución (12 bits), lo que es

posible configurando `pwm_set_wrap(slice, 4095)`.

III-F ¿Cómo se pueden mejorar las transiciones de color en el LED RGB?

Las transiciones abruptas entre colores pueden suavizarse implementando una interpolación lineal o una función de rampa entre el valor actual y el valor objetivo del duty cycle. En lugar de aplicar directamente el nuevo valor del potenciómetro, el código podría incrementar o decrementar el duty cycle actual en pequeños pasos con una temporización controlada, produciendo transiciones fluidas. Una mejora adicional sería aplicar corrección gamma, ya que la respuesta del ojo humano a la luminosidad es logarítmica y no lineal: los primeros niveles de PWM producen cambios de brillo más perceptibles que los últimos.

III-G ¿Qué aplicaciones prácticas tiene el uso de PWM en sistemas de control?

La modulación por ancho de pulso es una de las técnicas más utilizadas en ingeniería de control y electrónica de potencia. Sus aplicaciones más relevantes incluyen el control de velocidad en motores DC y trifásicos (variadores de frecuencia), la regulación de tensión en fuentes conmutadas (convertidores buck, boost y flyback), el control de posición en servomotores de modelismo y robótica, la iluminación LED regulable en sistemas de alumbrado inteligente, el control de temperatura mediante resistencias calefactoras, y la generación de señales de audio en amplificadores clase D de alta eficiencia.

III-H ¿Cuál es la diferencia entre PWM y PAM?

La **Modulación por Ancho de Pulso** (PWM) codifica la información variando el tiempo que cada pulso permanece en estado alto, manteniendo constante la amplitud y la frecuencia de la señal. La **Modulación por Amplitud de Pulso** (PAM, *Pulse Amplitude Modulation*) codifica la información variando la amplitud (altura) de cada pulso, manteniendo constante su duración. En microcontroladores digitales se prefiere el PWM porque los pines de salida solo pueden generar dos niveles fijos de tensión (alto o bajo), lo que hace que el PAM sea impracticable sin hardware analógico adicional como un DAC. El PWM, en cambio, se implementa puramente con lógica digital y contadores de hardware.

IV. CONCLUSIONES

- La combinación del ADC de 12 bits con el periférico PWM de 8 bits del RP2040 permitió construir un sistema de control de color completo con solo tres potenciómetros, sin componentes activos adicionales, demostrando el poder de los periféricos integrados del microcontrolador.
- La inversión del duty cycle ($255 - \text{duty}$) para el LED de ánodo común es un detalle crítico: sin ella, la lógica del control quedaría invertida (potenciómetro al máximo = LED apagado), evidenciando la importancia de verificar el tipo de LED (ánodo o cátodo común) antes de escribir el código.
- La impresión no bloqueante en el monitor serial mediante `to_ms_since_boot()` en lugar de `sleep_ms()` permitió mantener una actualización fluida del PWM cada 10 ms y al mismo tiempo enviar datos al monitor cada 500 ms, sin que ambas tareas interfirieran entre sí.
- La librería `hardware_pwm` del SDK abstrae completamente la manipulación de registros del periférico PWM del RP2040 (slices, canales, wrap, level), permitiendo configurar señales PWM en pocas líneas de código sin necesidad de conocer la arquitectura interna del microcontrolador.
- Como mejora futura se podría aumentar la resolución del PWM a 12 bits configurando `pwm_set_wrap(slice, 4095)` para aprovechar completamente la resolución del ADC, implementar corrección gamma para una respuesta de brillo perceptualmente lineal, o agregar un modo de ciclo automático de colores con transiciones suaves sin intervención del usuario.

ANEXOS

A. Código fuente – RGB.c

ANEXO A – RGB.c (completo)

```
#include "pico/stdlib.h"
#include "hardware/adc.h"
#include "hardware/pwm.h"
#include <stdio.h>

#define PIN_R 6 // GP6
#define PIN_G 10 // GP10
#define PIN_B 14 // GP14
#define ADC_R 0 // GP26
#define ADC_G 1 // GP27
#define ADC_B 2 // GP28
```



```

static void setup_pwm(uint pin) {
    gpio_set_function(pin, GPIO_FUNC_PWM);
    uint slice = pwm_gpio_to_slice_num(pin);
    ;
    pwm_set_wrap(slice, 255);
    pwm_set_enabled(slice, true);
}

static void set_pwm(uint pin, uint8_t duty)
{
    uint slice = pwm_gpio_to_slice_num(
        pin);
    uint channel = pwm_gpio_to_channel(pin);
    ;
    pwm_set_chan_level(slice, channel, duty);
}

int main() {
    stdio_init_all();
    adc_init();
    adc_gpio_init(26); adc_gpio_init(27);
    adc_gpio_init(28);
    setup_pwm(PIN_R); setup_pwm(PIN_G);
    setup_pwm(PIN_B);

    while (true) {
        adc_select_input(ADC_R); uint16_t
            raw_r = adc_read();
        adc_select_input(ADC_G); uint16_t
            raw_g = adc_read();
        adc_select_input(ADC_B); uint16_t
            raw_b = adc_read();

        uint8_t pct_r = (raw_r * 100) /
            4095;
        uint8_t pct_g = (raw_g * 100) /
            4095;
        uint8_t pct_b = (raw_b * 100) /
            4095;
        uint8_t duty_r = (raw_r * 255) /
            4095;
        uint8_t duty_g = (raw_g * 255) /
            4095;
        uint8_t duty_b = (raw_b * 255) /
            4095;

        set_pwm(PIN_R, 255 - duty_r);
        set_pwm(PIN_G, 255 - duty_g);
        set_pwm(PIN_B, 255 - duty_b);

        static uint32_t last = 0;
        uint32_t now = to_ms_since_boot(
            get_absolute_time());
        if (now - last >= 500) {
            last = now;
            printf("R: %d%% | G: %d%% | B:
                %d%%\n",
                pct_r, pct_g, pct_b);
        }
        sleep_ms(10);
    }
}

```

```

    }
    return 0;
}

```

B. Archivo CMakeLists.txt

ANEXO B – CMakeLists.txt (completo)

```

cmake_minimum_required(VERSION 3.13)
set(PICO_BOARD pico_w)
include(pico_sdk_import.cmake)

project(PRACTICA_12)

pico_sdk_init()

add_executable(RGB
    RGB.c
)

target_link_libraries(
    RGB
    pico_stdlib
    hardware_adc
    hardware_pwm
)

pico_enable_stdio_usb(1)
pico_enable_stdio_uart(0)

pico_add_extra_outputs(
    RGB
)

```

C. Diagrama en Protoboard

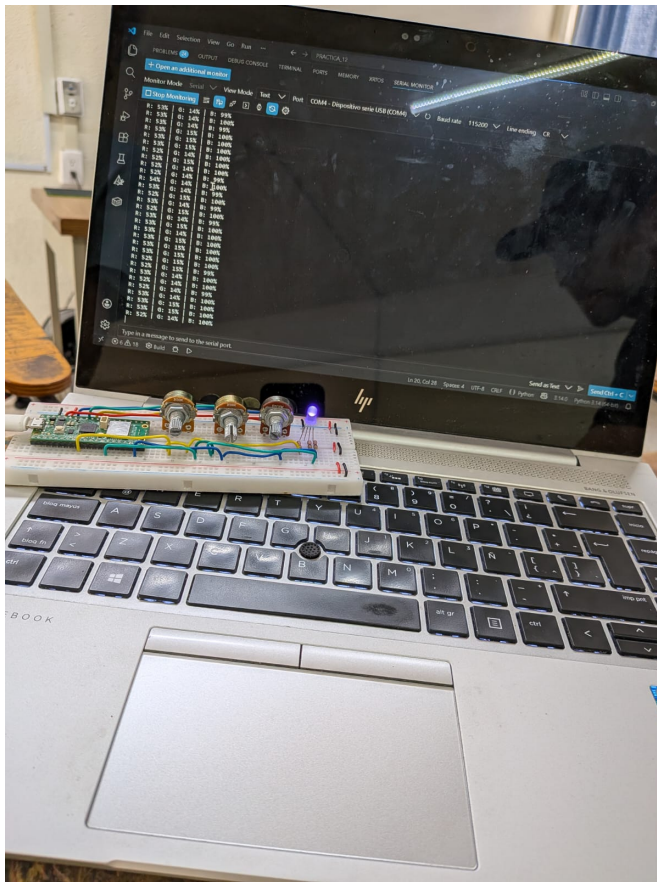


Figura 3: Diagrama en protoboard del circuito de control RGB con tres potenciómetros y LED RGB de ánodo común (Fritzing).